Page Replacement

Tags:

Time Limit: 1 s Memory Limit: 32 MB
Submission: 7 AC: 5 Score: 99.68

Submit Codes AI Code Tips

Description

Page replacement algorithms were a hot topic of research and debate in the 1960s and 1970s. That mostly ended with the development of sophisticated LRU approximations and working set algorithms. Since then, some basic assumptions made by the traditional page replacement algorithms were invalidated, resulting in a revival of research. In particular, the following trends in the behavior of underlying hardware and user-level software has affected the performance of page replacement algorithms:

Size of primary storage has increased by multiple orders of magnitude. With several gigabytes of primary memory, algorithms that require a periodic check of each and every memory frame are becoming less and less practical. Memory hierarchies have grown taller. The cost of a CPU cache miss is far more expensive. This exacerbates the previous problem.

Locality of reference of user software has weakened. This is mostly attributed to the spread of object-oriented programming techniques that favor large numbers of small functions, use of sophisticated data structures like trees and hash tables that tend to result in chaotic memory reference patterns, and the advent of garbage collection that drastically changed memory access behavior of applications.

Requirements for page replacement algorithms have changed due to differences in operating system kernel architectures. In particular, most modern OS kernels have unified virtual memory and file system caches, requiring the page replacement algorithm to select a page from among the pages of both user program virtual address spaces and cached files. The latter pages have specific properties. For example, they can be locked, or can have write ordering requirements imposed by journaling. Moreover, as the goal of page replacement is to minimize total time waiting for memory, it has to take into account memory requirements imposed by other kernel sub-systems that allocate memory. As a result, page replacement in modern kernels (Linux, FreeBSD, and Solaris) tends to work at the level of a general purpose kernel memory allocator, rather than at the higher level of a virtual memory subsystem.

There are many page replacement algorithms, one of them is LRU:

The least recently used page (LRU) replacement algorithm, though similar in name to NRU(Not recently used), differs in the fact that LRU keeps track of page usage over a short period of time, while NRU just looks at the usage in the last clock interval. LRU works on the idea that pages that have been most heavily used in the past few instructions are most likely to be used heavily in the next few instructions too. While LRU can provide near-optimal performance in theory (almost as good as Adaptive Replacement Cache), it is rather expensive to implement in practice. There are a few implementation methods for this algorithm that try to reduce the cost yet keep as much of the performance as possible.

One important advantage of LRU algorithm is that it is amenable to full statistical analysis. It has been proved, for example, that LRU can never result in more than N-times more page faults than OPT algorithm, where N is proportional to the number of pages in the managed pool.

The most expensive method is the linked list method, which uses a linked list containing all the pages in memory. At the back of this list is the least recently used page, and at the front is the most recently used page. The cost of this implementation lies in the fact that items in the list will have to be moved about every memory reference, which is a very time-consuming process.

Another method that requires hardware support is as follows: suppose the hardware has a 64-bit counter that is incremented at every instruction. Whenever a page is accessed, it gains a value equal to the counter at the time of page access. Whenever a page needs to be replaced, the operating system selects the page with the lowest counter and swaps it out. With present hardware, this is not feasible because the required hardware counters do not exist.

The LRU algorithm with 3 pages at most in the management pool runs as follows:

```
7  0  1  2  0  3  0  4  2  3  0  3  2  1  2  0  1  7  0  1  reference string
----------------------------------------------------------------------------------------
-
7  7  7  2     2     4  4  4  0        1     1     1
   0  0  0     0     0  0  3  3        3     0     0       page frames in pool
      1  1     3     3  2  2  2        2     2     7
```

12 page faults occurred.

For a given reference string, you need to calculate the number of page faults.

Input

The first line contains an integer, the number of test cases. Each test case contains two lines, the first line is the capacity of the management pool m(0 < m ≤ 10000), and the length of reference string n(0 < n ≤ 100000). The next line contains exactly n integers, which indicate the reference sequence of page frames (page number ranged from 0 to n).

Output

For each test case, the output should contains the number of page faults that occurred.

Samples

input Copy

```
3
3 5
1 2 3 4 5
3 5
1 2 1 2 3
3 20
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
```

output Copy

```
5
3
12
```

Source

湖南大学2008校赛

Submit Codes

HZNUOJ is based on HUSTOJ

--- Maintainer List ---

Server Time: 2025-8-26 15:12:04